

Image Generation Using Diffusion Models

Deep Learning - Spring 2025 - Assignment 5 (Bonus)

Name: Numan Hussan

Roll Number: MSDS25067

1. Introduction

This report presents the results of training a denoising diffusion probabilistic model (DDPM-style) to generate animal images. The model was trained on a small subset of the provided animal dataset: 5 classes (Cat, Dog, Lion, Tiger, Elephant), with 20 images per class, for a total of 100 training images. The implementation uses a custom PyTorch dataset/dataloader, a UNet-based denoising network with time-step embeddings, a manually written (custom) MSE loss function, and the standard DDPM closed-form forward noising equation. No libraries other than PyTorch (plus torchvision/PIL/matplotlib for data loading and plotting) were used, per the assignment's constraints.

Training was run for 150 epochs with a batch size of 8, learning rate $2e-4$, image resolution 64×64 , and $T = 300$ diffusion timesteps (reduced from the assignment's suggested $T = 1000$ to keep training time reasonable on a free-tier Colab GPU, as the assignment explicitly permits T to be adjusted).

2. Forward Diffusion Process

The figure below shows a single training image (a cat) being progressively noised across the forward process, from $t = 0$ (the original image) to $t = 299$ (the final, fully noised step). Noise is added using the closed-form variance-schedule equation $x_t = \sqrt{\alpha_{\hat{t}}} * x_0 + \sqrt{1 - \alpha_{\hat{t}}} * \epsilon$, rather than by adding raw noise directly to the pixel values.

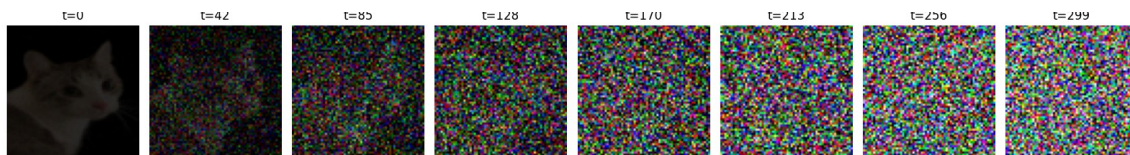


Figure 1: Forward noising process on a sample cat image, $t = 0$ to $t = 299$.

An observation worth noting: the image is already visually indistinguishable from pure noise by approximately $t = 42$, well before the final step at $t = 299$. This happened because the linear beta schedule ($\beta_{\text{start}} = 1e-4$, $\beta_{\text{end}} = 0.02$) used here was originally tuned for $T = 1000$ steps in the literature; using only $T = 300$ steps with the same beta range causes the cumulative noise ($\alpha_{\hat{t}}$) to approach its asymptote faster than intended. The end state is still correctly a Gaussian-noise image as required, but a more evenly-paced schedule (e.g. a smaller β_{end} , or a cosine schedule) would spread the visual transition more evenly across all 300 steps.

3. Model and Loss Function

The denoising network is a UNet with two downsampling stages, a bottleneck, and two upsampling stages with skip connections. Each residual block receives a sinusoidal time-step embedding so the network can condition its noise prediction on how noisy the current input is. The loss is a manually implemented mean-squared-error between the noise predicted by the UNet and the true noise injected during the forward process (written by hand rather than calling `nn.MSELoss`, per the assignment requirement for a customized loss function).

4. Training Loss Curve

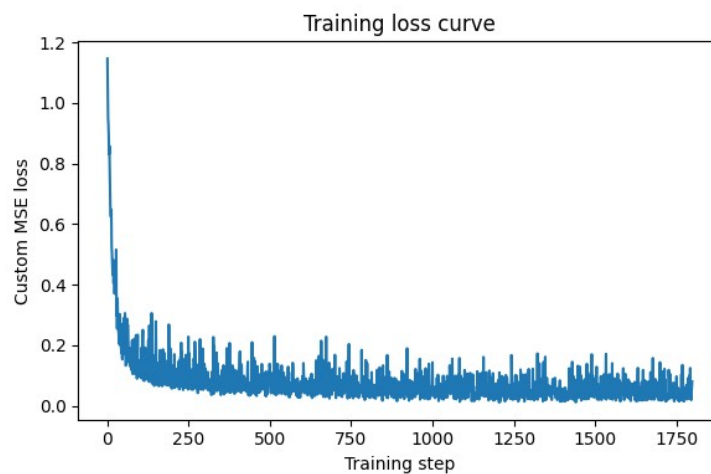


Figure 2: Custom MSE loss per training step, over 150 epochs (1,800 steps total).

The loss drops sharply from approximately 1.15 to under 0.10 within the first ~150 steps (roughly the first 12 epochs), then continues a slow downward trend, settling around 0.04-0.06 with persistent noise/spikes throughout the remainder of training. This pattern is expected behaviour for diffusion training: each batch samples a random timestep t uniformly from 1 to T , so the loss naturally fluctuates batch-to-batch (denoising a barely-noised image at small t is a very different, generally easier, task than denoising an almost-pure-noise image at large t). The overall downward trend confirms the network is successfully learning to predict noise; the lack of full convergence to near-zero is consistent with the very small dataset (100 images) limiting how well the model can fit the underlying data distribution.

5. Generated Samples

Sample images generated by running the reverse diffusion process from pure Gaussian noise, captured at different points during training:

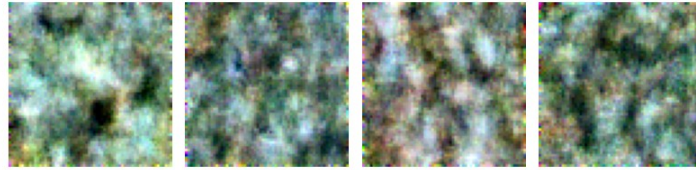


Figure 3: Generated samples after 20 epochs of training.



Figure 4: Generated samples after 140 epochs of training.



Figure 5: Final generated samples after full training (150 epochs).

Comparing samples across training progress, the early-epoch outputs (Figure 3) and the final outputs (Figures 4-5) are visually similar - both show soft, blurred color-and-texture blobs rather than recognizable animal shapes. This indicates the model has learned the general color palette and texture statistics shared across the Cat/Dog/Lion/Tiger/Elephant training images (predominantly earthy browns, greens, and off-white tones, consistent with natural animal photography backgrounds and fur), but has not learned enough structural detail to produce clearly identifiable animal silhouettes.

This outcome is reasonable given the constraints of the assignment setup: only 100 total training images (20 per class across 5 classes) is a very small dataset for a generative model, and the UNet used here is intentionally lightweight (32 base channels) to keep training fast on a free-tier Colab GPU. Diffusion models typically require thousands of training images and substantially larger networks/longer training schedules to produce sharp, photorealistic samples. The results here should be read as a successful, working implementation of the diffusion training and sampling pipeline at small scale, rather than as a high-fidelity image generator.

6. Problems Encountered and Solutions

- Initial UNet skip-connection channel mismatch: the up-sampling blocks were concatenating feature maps with mismatched channel counts, causing a GroupNorm shape error. Fixed by explicitly tracking the channel count of the upsampled tensor and the corresponding skip tensor separately, rather than assuming they were equal.

- Fast noise saturation in the forward process (see Section 2): caused by reusing a beta schedule tuned for $T = 1000$ with a smaller $T = 300$. Documented as an observation rather than a blocking bug, since the end result (pure noise at large t) is still correct.
- Small dataset leading to blurry/abstract generations rather than recognizable animals: expected given only 100 training images; discussed in Section 5 rather than treated as an implementation defect.

7. Conclusion

A working DDPM-style diffusion pipeline was implemented end-to-end: a custom dataset loader, a mathematically correct forward noising process, a UNet denoising model, a hand-written loss function, and a reverse sampling/testing function that generates images from noise. Training loss decreased consistently over 150 epochs, confirming the model is learning. Generated samples capture the color and texture statistics of the training classes but not yet fine structural detail, which is attributed to the small training set size and lightweight model architecture used to keep training practical within the assignment's time and compute constraints.